



FACULTY
OF ELECTRICAL
ENGINEERING
CTU IN PRAGUE

A Lightweight Noise-Robust Post-Trigger Classifier for Embedded Acoustic Artillery Launch Detection

Martin Maxa Jakub Svatoš Jakub Zelinka

Czech Technical University in Prague

Faculty of Electrical Engineering, Department of Measurement

BEC 2026 – Baltic Electronics Conference

Tallinn, Estonia · October 2026

Motivation – decide on the edge, in noise

- Passive acoustic sensing supports artillery early warning: it is **cheap**, needs **little power** and **emits no signal** – active systems are expensive, power-hungry and easier to locate.

The decision has to be on the edge

- Cheap nodes mean many nodes, spread over a large area.
- Streaming raw audio from every node costs power and radio bandwidth – and a transmitting node is easier to find.
- So each node decides by itself and sends an **alert**, not the audio.

The signal arrives weak

- Artillery fires from far away – in our recordings the guns were $\approx$ **5–10 km** from the station.
- The launch reaches the sensor attenuated, on top of wind, traffic and other background.
- So the classifier has to work at **low SNR**.



low SNR

Q1 Where to add the training noise?

Can cheap **MFCC-domain** augmentation replace **waveform-domain** noise augmentation for a compact post-trigger classifier?

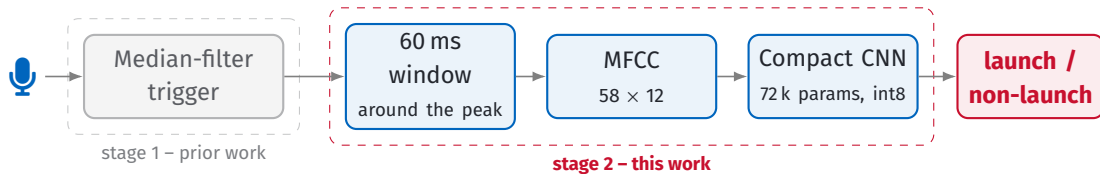


on the edge

Q2 Does it survive the chip?

Does the resulting robustness survive **int8 quantization** and deployment on an **MCU** (ESP32-S3)?

Two-stage detection pipeline



Stage 1 – impulse detection

- Runs continuously on the stream: signal $\rightarrow$ energy (x^2), a **median filter** over neighbouring short windows estimates the local background.
- Background subtracted; a peak is a candidate only if it passes adaptive amplitude, RMS-ratio and energy tests and **rises like an impulse**.
- Detected all tested small-arms shots, reliable down to **5 dB** SNR (Svatoš & Holub, RTSI 2019).

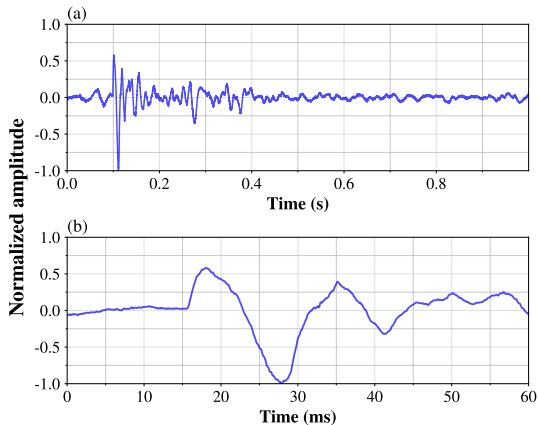
Stage 2 – classification

- Runs only on the candidates – peaks that look like an acoustic impulsive event.
- The trigger also fires on doors, shouts, small arms ... – a **binary** classifier rejects them.
- Launch / non-launch: stage 2 is in effect the **artillery launch detector**. We evaluate this stage only.

Field recordings



- Two days, one **152 mm** self-propelled howitzer.
- Station 1-2 km from the impact area; guns $\approx$ 5-10 km away.
- 4 synchronized mics (PreSonus PRM1), Roland Rubix 44; 16 bit, 22.05 kHz.



(a) launch recording; (b) the 60 ms analysis window cut around the detected peak.

Labelled corpus and event-level split

Subset	Events	Samples		
		Train	Val.	Test
Artillery launch	50	128	8	10
Impulsive noise	706	457	109	140
Small-arms shots	85	49	18	18
Total	841	634	135	168

- Non-launch: false-trigger candidates – door impacts, shouts, bells, **small-arms gunshots** as hard negatives.
- Split **by event**: the four channels of one shot never cross partitions.
- Training uses all 4 channels (launch share 20 %, class-weighted loss); validation and test use one.

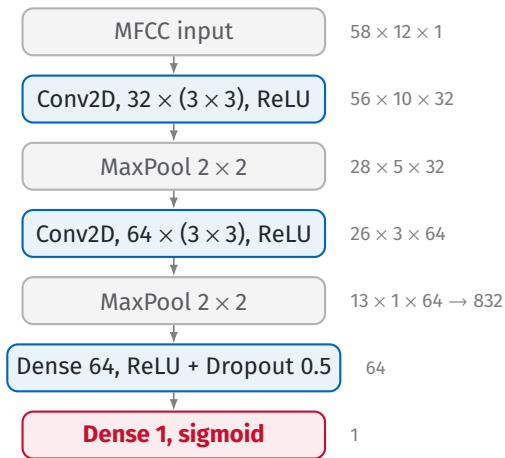
MFCC front end and compact CNN

MFCC

- 2.5 ms frames, 1 ms hop, Hamming
- 512-point FFT, 40 mel filters
- first 12 coefficients
- input tensor 58×12

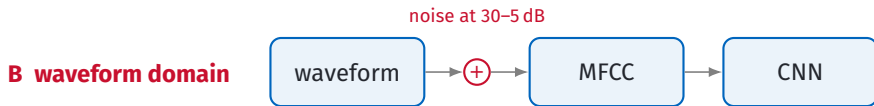
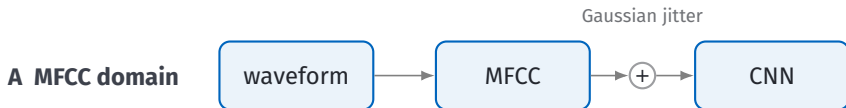
Training

- Adam, binary cross-entropy
- early stopping on validation loss



72 193 trainable parameters

Two ways to train for noise



- A is cheap: the MFCCs are computed once, noise is added to the matrix.
- But additive noise *before* MFCC extraction distorts the cepstrum **nonlinearly** and signal-dependently – A may not imitate it.
- Augmentation on the training split only.

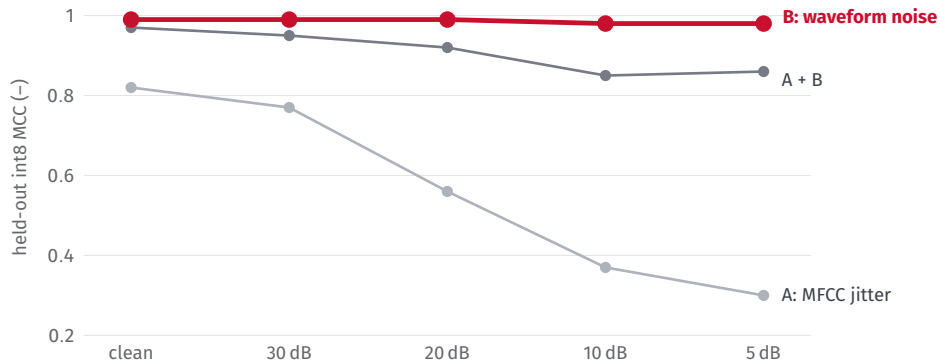
- Held-out test partition, used only for the final numbers.
- Noise added to the **waveform before MFCC extraction**, per event:

$$\sigma_n^2 = P_x \cdot 10^{-\text{SNR}/10}$$

at **30, 20, 10, 5 dB** – plus 0 and –5 dB as a stress test, beyond the trigger's 5 dB envelope.

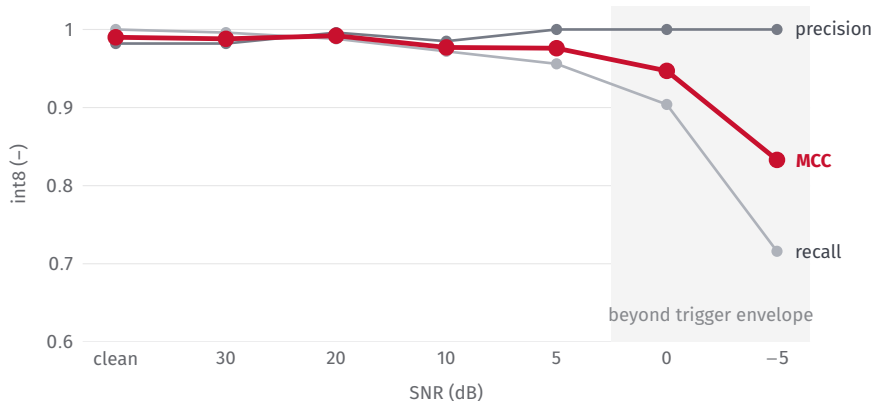
- Peak detection and windowing re-run on the noisy signal.
- **5 training seeds** × **5 noise seeds** = 25 runs per cell, mean ± std; threshold 0.5.
- Three regimes: **float32** and **int8** on the PC, int8 on the **ESP32-S3**.
- Main metric: Matthews correlation coefficient (MCC).

Result 1 – where to add the noise



Same network, three augmentation strategies; mean over 25 runs. At 5 dB: A 0.30 ± 0.21 , A + B 0.86 ± 0.07 , B 0.98 ± 0.04 .

Result 2 – held-out robustness of the deployed model



Precision stays **1.000** from 5 dB down – in heavy noise the model misses launches, it does not raise false alarms. float32 vs. int8: within ± 0.02 MCC, no consistent direction.

Result 3 – on the ESP32-S3

Same answers as the PC

1 171 of 1 176 inferences bit-exact with PC int8.

The 5 others: scores at exactly 0.5 – a legal one-bit rounding difference between TFLite and TFLite Micro kernels.

Latency

32.0 ms per MFCC segment (31.99–32.09 ms), independent of noise.

Classifier inference only – MFCC computed on the host.

TensorFlow Lite Micro, int8

Model size	81 400 B
tensor_arena	80 KiB
Flash – code	201.5 kB
Flash – data	144.0 kB
RAM	143.8 kB

All 8 operators native in TFLite Micro; compute-heavy ones
via ESP-NN kernels.

- **Stage 2 only** – no continuous-stream false-trigger rate; results assume the trigger found the peak.
- **Small positive class** – 50 launch events, one gun type (152 mm); other calibres untested.
- **Synthetic Gaussian noise** – closer to reality than feature jitter, but no field noise, propagation or device shift.
- **One event-level split**; no session-independent evaluation.
- **MFCC on the host** – on-device MFCC, end-to-end latency and energy per decision are future work.

- A compact MFCC-CNN (72 k parameters) works as the post-trigger stage of an embedded artillery launch detector.
- **Add the noise to the waveform, not to the features:** MFCC-domain jitter collapses at 5 dB (MCC 0.30) and even hurts when combined.
- Waveform-trained model: **MCC 0.99 at 30 dB, 0.98 at 5 dB**; precision 1.0 under heavy noise.
- int8 is performance-neutral; ESP32-S3 matches the PC, 32 ms per inference.

Thank you – questions?

maxamart@fel.cvut.cz

Appendix – full held-out robustness table

SNR (dB)	Accuracy (%)	Precision	Recall	F1	MCC	MCC (float32)
Clean	99.88 ± 0.24	0.982 ± 0.037	1.000 ± 0.000	0.990 ± 0.019	0.990 ± 0.020	1.000 ± 0.000
30	99.86 ± 0.26	0.982 ± 0.037	0.996 ± 0.020	0.988 ± 0.021	0.988 ± 0.022	0.998 ± 0.011
20	99.90 ± 0.22	0.996 ± 0.018	0.988 ± 0.033	0.992 ± 0.019	0.992 ± 0.020	0.993 ± 0.018
10	99.74 ± 0.39	0.985 ± 0.034	0.972 ± 0.061	0.977 ± 0.035	0.977 ± 0.035	0.974 ± 0.039
5	99.74 ± 0.42	1.000 ± 0.000	0.956 ± 0.071	0.976 ± 0.039	0.976 ± 0.039	0.978 ± 0.036
0 ^a	99.43 ± 0.53	1.000 ± 0.000	0.904 ± 0.089	0.947 ± 0.049	0.947 ± 0.049	0.934 ± 0.045
-5 ^a	98.31 ± 1.01	1.000 ± 0.000	0.716 ± 0.170	0.823 ± 0.122	0.833 ± 0.107	0.820 ± 0.088

Deployed int8 classifier, mean ± std over 5 training and 5 noise seeds. Test partition: 168 samples, 10 of them launches.

^aBeyond the trigger's validated 5 dB envelope.

Appendix – design choices

Why supervised, not anomaly detection?

- After the trigger, *every* candidate is already anomalous against the background – small-arms shots included.
- One-class SVM / isolation forest / autoencoder would flag a rifle shot as “anomalous” – not the decision we need.
- They fit stage 1 (novelty detection) rather than stage 2.

Why MFCC, not GTCC?

- GTCC won in our gunshot study (IMEKO 2026); MFCC is standardized with mature embedded implementations.

Why 22.05 kHz?

- Launch energy lies well below the 11 kHz Nyquist limit.
- Buffers and MFCC compute scale with f_s – 44.1 kHz would spend RAM without adding usable signal.

Why ESP32-S3?

- Low cost, low power, widely available; ESP-NN accelerated kernels for TFLite Micro.
- A representative Edge-AI MCU – not a claim about all embedded targets.

Appendix – the five ESP32 disagreements

- All five: PC int8 score is **exactly 0.5** – on the decision threshold.
- The model is very confident, so the output quantization is coarse: **one LSB spans 10.8 logit units**.
- TFLite and TFLite Micro kernels may round one LSB differently – both legal.
- Platform behaviour is equivalent up to rounding at the boundary; 1 171 / 1 176 bit-exact.

Precision share of predicted launches that are real launches (false alarms).

Recall share of real launches that were detected (missed launches).

MCC Matthews correlation coefficient – uses the whole confusion matrix, robust to class imbalance; +1 perfect, 0 random.

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

Test partition: 10 launches vs. 158 non-launch – accuracy alone would be 94 % for a model that never says “launch”.